

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS750 – Instructional Design Document

Forest of Keys

Preparing COS212 Students for Binary Search Trees

Group Members
Ntsako Sithole (u26839628)
Mutarisi Chibaya (u23546094)
Jared Hürlimann (u23543932)
Goodman Mawelele (u17127182)
Thendo Mudau (u21529257)

Data structure: Binary Search Tree (BST)
Algorithm: BST Search

Summary

Forest of Keys is a browser-based learning application that prepares COS212 students for a lecture on binary search trees (BSTs). It focuses on one data structure, that is the binary search tree and one algorithm which is the BST search. The application teaches students how to identify the parts of a tree, recognise the BST ordering property, trace searches for keys that are present or absent, count comparisons, and explain how tree shape affects search effort.

A BST is a tree in which each node has at most two children. For distinct keys, smaller values are placed in the left subtree and larger values in the right subtree. During a search, each comparison determines whether the next step is to move left or right. This allows the search to discard the subtree that cannot contain the target. If the target is not found before the search reaches an empty position, the key is absent.

Scope and design decisions

The application assumes no prior knowledge of trees. Students move from labelled examples and demonstrations to partially completed activities, independent practice, and a final readiness check. The app gives explanatory feedback and suggests what to revise. Activities are ungraded, retries are available, and learning activity remains in the browser.

The algorithm is BST search, not a full treatment of all BST operations. BST insertion is used only in the sandbox to let students create different tree shapes and investigate their effect on search paths; insertion is not taught or assessed as a second algorithm. The design therefore removes the draft's separate insertion lesson, insertion mastery targets, delete-node functionality, and traversal simulator as core learning features. The sandbox may still use insertion as a tree-building interaction.

The draft's badges, XP, certificate, timed challenges and drag-and-drop exercises are also removed. Students make every search decision with *Go left*, *Go right* and *Found it* buttons, which work with a keyboard and on a phone, and motivation comes from streaks, visible progress and outcome-level feedback rather than rewards.

The design model is ADDIE. The instructional sequence uses gradual release of responsibility, supported by Gagné's nine events and Merrill's First Principles of Instruction. Learning activities also draw on constructivist, cognitive, behavioural, and social-cognitive principles where they directly support the stated outcomes.

Learning outcomes

Measurable learning outcomes

- After using Forest of Keys, a student will be able to:

LO1: Tree vocabulary

- Name and identify the root, parent, child, leaf, and subtree on a labelled tree diagram.

LO2: BST property

- State the BST ordering rule and decide whether a given tree satisfies it, including the ordering constraints throughout each subtree.

LO3: Search tracing

- Trace a BST search and list the nodes visited for both a key that is present and a key that is absent.

LO4: Comparison counting

- Predict the number of key comparisons required to search for a given key in a displayed tree.

LO5: Tree shape and efficiency

- Explain how balanced and lopsided tree shapes affect the number of comparisons, and explain why inserting keys in sorted order can produce a lopsided tree.

LO6: Readiness reflection

- Interpret the outcome-level results of the final check and select an appropriate revision route.

Instructional design model and learning theories

ADDIE:

- **Analysis:** The application is intended for COS212 students preparing for an introductory lecture on BSTs. It assumes no prior knowledge of tree terminology or BST search. The key learning need is to understand the structure, ordering rule, and search process before the lecture.
- **Design:** The six learning outcomes guide the lesson sequence, practice activities, feedback, and final check. Content is sequenced from vocabulary and the BST rule to search tracing, comparison counting, tree-shape exploration, and readiness reflection. Assessment items are mapped to the relevant outcomes.
- **Development:** The web app contains labelled tree diagrams, step-by-step search animations, iterative pseudocode, generated practice questions, a sandbox, and an outcome-level results screen. Explanations are always available as text, with optional narration and visual demonstrations.

- **Implementation:** Students access the app in a browser, or install it on a phone, and work through the activities at their own pace. It works offline after the first visit. The app provides hints, retries, and feedback. Nothing is marked, and learning activity remains in the browser.
- **Evaluation:** Formative checks in every lesson and in practice identify misconceptions as they happen. The final check reports performance by learning outcome and recommends a revision route. The design can be improved using student feedback and observed difficulty patterns.

Learning Theories Applied

Constructivism

- Students build their own BSTs in the sandbox by inserting keys, and see how the order of insertion shapes the tree.
- BST search is introduced through a higher-or-lower guessing game, linking to prior knowledge of binary search and comparisons: a BST is "binary search turned into a structure".
- Activities ask for a prediction before the answer is revealed (e.g., "Where does 35 belong: the left or the right subtree of 50?", "Is this tree a valid BST?").
- The sandbox invites discovery: "Try inserting 1, 2, 3, 4, 5 in that order. What happens?" Students see the lopsided chain form before it is explained.

Cognitivism / Information Processing

- Content is chunked into ten short screens, each split into one or two small parts.
- Dual coding: every rule is shown as text next to a tree diagram, and the search is animated as the key travels down the tree.
- Worked examples precede practice, reducing cognitive load.
- Signalling: the current node is highlighted, the comparison (<, > or =) is shown, and the matching line of pseudocode lights up.
- Every technical term can be tapped for a plain-language definition, and a glossary is always available.

Behaviourism

- Immediate feedback: correct answers are marked in blue with ✓, incorrect answers in orange with ✗ and an explanation. Colour is never used alone, and blue/orange is distinguishable with colour blindness.
- Practice streaks, a best streak and growing levels (Seedling, Sapling, Old oak) reward consistent success.
- Repeated practice: two correct answers in a row move the student up a difficulty level, and a wrong answer moves them down one.

Social Cognitive Theory

- "Watch me" mode demonstrates search step by step, for a key that is present and a key that is absent.
- Students observe worked examples, then practise with guidance: the app performs the first steps and hands over more decisions with each problem.
- Progress tracking (the journey progress bar, ticks on completed phases, and comparison with the previous attempt) builds self-efficacy.
- Confidence grows through small successes and unlimited retries.

Catering to Different Learning Styles

Fleming's VARK Model

- Visual: Animated BST diagrams, highlighted search paths, and lopsided and balanced trees shown side by side.
- Auditory: Optional narration. Lessons have *Listen* buttons and the worked example reads each step aloud, using the device's own voice so that nothing is sent over the network.
- Read/Write: Pseudocode, plain-language explanations, a glossary, and a summary sheet that can be printed or downloaded.
- Kinesthetic: Walking each search with *Go left* / *Go right* / *Found it* buttons, tapping nodes to identify their roles, the guessing game, and the interactive sandbox.

Kolb's Learning Cycle & Honey & Mumford Styles

- Concrete Experience (Activist) → Play the higher-or-lower game and place keys in a growing tree.
- Reflective Observation (Reflector) → Prompts like "Try inserting 1, 2, 3, 4, 5 in that order. What happens?" and "Why does sorted order make a chain?"
- Abstract Conceptualisation (Theorist) → Formal BST rule, search rules and iterative pseudocode.
- Active Experimentation (Pragmatist) → Real-world task: build a BST of student numbers, search it, and compare it with the same numbers inserted in sorted order.

Motivation Strategies

Flow Theory

- Start simple (3-node BST), then scale up difficulty.
- Clear goals (the outcomes are listed on the Welcome screen) and immediate feedback maintain engagement.

- Adaptive difficulty in practice keeps a balance between challenge and skill.

Malone & Lepper's Framework

- Challenge: Practice levels increase in complexity; trick cases such as trees that break the rule only deeper in a subtree.
- Curiosity: Prompts like "What happens if we insert numbers in sorted order?"
- Control: Students choose their pace, can skip or retry, pick sandbox values and animation speed, and adjust theme, text size, motion and narration.
- Fantasy: Themed scenario ("Forest of Keys", with practice levels Seedling, Sapling and Old oak).
- Recognition: Streaks, ticks on completed phases, and a "Lecture-ready" result.

Keller's ARCS Model

- Attention: Opening question ("How does your phone find one contact among 1000 in a split second?"), animations, varied activities.
- Relevance: Links to real-world uses (databases, file systems, and Java's TreeMap, which students use in COS212) and the class-list task.
- Confidence: Clear objectives, hints, fading support, unlimited retries, and nothing is marked.
- Satisfaction: Progress bar, an outcome-by-outcome report, the "Lecture-ready" result, and a summary sheet to take to the lecture.

Assessment and Scaffolding

Starting point

- No separate diagnostic pre-test is used, because the application assumes no prior knowledge of trees.
- Students who already know BSTs can go straight to the final check from the Welcome screen, and use its results to decide what to revisit.

In-Lesson Checks

- Each learning screen includes short activities with instant feedback: three node-identification tasks (Screen 1), four key predictions and three valid-or-invalid trees (Screen 2), the guessing game (Screen 3), and four searches with comparison counting (Screen 5).
- In guided searches, a first wrong move gives a hint; a second wrong move on the same step gives the full explanation and the correct move.

Practice

- Mixed generated questions on LO2–LO5 at three difficulty levels, with explanatory feedback after every answer but no step-by-step hints.

Final Check

- 10 questions, two for each content outcome (LO1–LO5), generated afresh on every attempt. Feedback is held back until the end so students can focus. Questions are tagged to Bloom's levels:
    - Remember: "Which node is the root?", or identify a leaf, parent or subtree (LO1).
    - Understand: "Is this tree a valid BST?" (LO2).
    - Apply: Trace a search by walking the key down the tree, and count its comparisons (LO3, LO4).
    - Analyse: Compare a balanced and a lopsided tree with the same keys and explain why one is taller and needs more comparisons (LO5).

Scaffolding Rules

- 80% or more: Lecture-ready. Skim any outcome that is not yet secure.
- 50–79%: Review the outcomes marked for revision, then retake the check.
- Below 50%: Guided route: return to the worked example and "Your turn", which have hints, then retake the check.
- Each outcome is reported as secure (✓), partly (~) or revise (✗) with a link back to its lesson, and the student chooses which outcome to revise first (LO6). Every answer can be reviewed with its explanation.

User Journey

Screen 0: *Welcome*

- Hook: "How does your phone find a contact among 1000 in a split second?"
- Learning outcomes listed, reassurance: nothing is marked, retries are free
- Shows where BSTs are used in practice: databases, file systems and Java's TreeMap.
- Purpose: Introduce the application, set expectations and reduce anxiety. The learning outcomes are briefly introduced in plain language.

Screen 1: *Introductory lesson*

- "What is a tree?"
- Views a labelled tree and selects terms to see what they mean
- Purpose: teach the basic vocabulary for LO1

Screen 2: *The BST rule*

- Learns that smaller values belong in the left subtree and larger values in the right subtree.
- Predicts where values belong and compares valid and invalid trees.
- Purpose: Develop understanding of the BST property (LO2), including the fact that it applies throughout a subtree, not only to direct children.

Screen 3: *How a search works*

- Uses a familiar higher-or-lower guessing analogy before following the search rules and iterative pseudocode.
- Purpose: Introduce the search process that students will apply when tracing a search (prepares for LO3).

Screen 4: *Worked example*

- Watches the application search for a key one step at a time. Each comparison is explained, the current node is highlighted, and an empty position is shown when the key is absent.
- Purpose: Demonstrate the search process and comparison counting (LO3 and LO4).

Screen 5: *Your turn*

- Completes partially worked search problems. Initially, the application provides some of the steps.
- The student gradually takes responsibility for more decisions until they can trace the full search independently.
- Purpose: Practice tracing searches for present and absent keys with decreasing support (LO3 and LO4).

Screen 6: *Practice*

- Answers mixed generated questions without step-by-step hints and receives explanatory feedback.
- The difficulty can change in response to performance.
- Purpose: Provide independent practice across the main content outcomes (LO2–LO5).

Screen 7: *Sandbox*

- Enters keys to build a tree and explores the search paths. The student can try inserting 1, 2, 3, 4, 5 and compare the resulting lopsided tree with a more balanced example
- Real-world task: builds a class list of student numbers, searches it, then compares it with the same numbers inserted in sorted order.

- Purpose: Explore how tree shape affects search comparisons (LO5). Insertion is used only to create trees for exploration, not taught as a second algorithm.

Screen 8: *Final Check*

- Answers 10 questions selected to sample the five content outcomes, LO1–LO5.
- Once the check has started, leaving it through the menu asks for confirmation, so answers are not lost by accident.
- Purpose: Check the student's current understanding of the content outcomes.

Screen 9: *Results*

- Views an overall result, an outcome-level report and a suggested revision route. An optional summary is available to print or download.
- Purpose: Support the student's reflection on readiness and choice of what to revise (LO6).

App Features Summary

- **Search walker:** Watch, guided and independent modes with *Go left* / *Go right* / *Found it*, a highlighted path and live pseudocode.
- **Sandbox:** Insert typed or preset keys with an animated walk-down, search any key, compare lopsided and balanced shapes, and the class-list task. Insertion builds trees only; there is no delete.
- **Adaptive practice:** Generated questions at three levels, with streaks and explanatory feedback.
- **Final check and results:** 10 questions, an outcome-level report, a revision route, a review of every answer, and a printable or downloadable summary.
- **Glossary:** Every technical term can be tapped for a definition, and a full glossary panel is always available.
- **Accessibility and comfort:** Colour-blind-safe colours always paired with symbols, light and dark themes, three text sizes, reduced motion, optional narration, keyboard support and a highly legible font.
- **Privacy and access:** No login and no back end; progress is saved only in the student's browser. The app can be installed on a phone and works offline.